DEDICATED TO INNOVATION

www.bacancy.com

BACANCY®

Customer Satisfaction Is Our Highest Priority

Employee Matters

In this course

What we are learning in this session

- Views
- Functions
- Stored Procedures
- Indexes
- Joins
- Create Database
- Create Table

Let’s Start

SQL Server - Views

- In SQL Server, a view is a virtual table whose values are defined by a query. In another word, a view is a name given to a query that can be used as a table. The rows and columns of a view come from tables referenced by a query.

- **Types of Views:**
  1). User-defined Views
  2). Indexed Views
  3). Partitioned Views
  4). System Views

- **Syntax:**

```
CREATE VIEW <schema_name>.<view_name>
AS
SELECT column1, column2, ...
FROM table1, table2,...
[WHERE];
```

- **Important Points:**
  - Unless indexed, a view does not exist as a stored set of data values in a database.
  - Views can be created by using tables or other views in the current or other databases.
  - The SQL statements comprising the view are stored in the database and not the resulting data.
  - The data from a view is generated dynamically when a view is referenced.
  - Views are used as a security mechanism to mask the underlying base tables and permit user access only to the view.

SQL Server - Functions

- Functions in SQL Server are similar to functions in other programming languages. Functions in SQL Server contains SQL statements that perform some specific tasks. Functions can have input parameters and must return a single value or multiple records.
- If your scripts use the same set of SQL statements repeatedly then this can be converted into a function in the database.
- **Types of Functions:**
  SQL Server Functions are of two types.

  1). System Functions: These are built-in functions available in every database. Some common types are Aggregate functions, Analytic functions, Ranking functions, Rowset functions, Scalar functions.

  2). User Defined Functions (UDFs): Functions created by the database user are called User-defined functions. UDFs are of two types:

SQL Server - Functions

2.1). Scalar functions: The function that returns a single data value is called a scalar function.

2.2). Table-valued functions: The function that returns multiple records as a table data type is called a Table-valued function. It can be a result set of a single select statement.

- **Advantage of User-defined Functions:**
  - Faster Execution: Similar to stored procedures, UDFs reduce the compilation cost of T-SQL by caching the plans and reusing them for future executions.
  - Reduce Network Traffic: The SQL statements of a function execute in the database, and the application calling it needs to make a function call to the database.
  - Supports Modular Programming: UDFs can be modified independently of the application source code. You can create UDFs once and store them in the database, and they can be called any number of times.

SQL Server - Stored Procedures

- In SQL Server, a stored procedure is a set of T-SQL statements which is compiled and stored in the database. The stored procedure accepts input and output parameters, executes the SQL statements, and returns a result set if any.
- By default, a stored procedure compiles when it gets executed for the first time. It also creates an execution plan that is reused for subsequent executions for faster performance.
- **Stored procedures are of two types:**
  1). User-defined procedures: A User-defined stored procedure is created by a database user in a user-defined database or any System database except the resource database.

  2). System procedures: System procedures are included with SQL Server and are physically stored in the internal, hidden Resource database and logically appear in the sys schema of all the databases. The system stored procedures start with the sp_ prefix.
- **Create Stored Procedure:**
  Use the CREATE statement to create a stored procedure.

SQL Server - Stored Procedures

Syntax:

```
CREATE [OR ALTER] {PROC | PROCEDURE} [schema_name.] procedure_name([@parameter
data_type [ OUT | OUTPUT | [READONLY]] [ WITH <procedure_option> ]
[ FOR REPLICATION ]
AS
    BEGIN
        sql_statements
    END
```

Advantages of Stored procedures:

- Stored procedures are reusable. Multiple users in multiple applications can use the same Stored Procedure.
- SPs reside in the database, it reduces network traffic. Applications have to make a procedure call to the database and it communicates back to the user.

SQL Server - Stored Procedures

- **Advantages of Stored procedures:**
  - Database objects are encapsulated within a stored procedure, and this acts as a security mechanism by restricting access to the database objects. Reduced development cost, easily modified, and increased readability.
  - Improves performance. When a stored procedure is executed for the first time, the database processor creates an execution plan which is reused every time this SP is executed.

SQL Server - Indexes

- An Index in SQL Server is a data structure associated with tables and views that helps in faster retrieval of rows.
- Data in a table is stored in rows in an unordered structure called Heap. If you have to fetch data from a table, the query optimizer has to scan the entire table to retrieve the required row(s). If a table has a large number of rows, then SQL Server will take a long time to retrieve the required rows. So, to speed up data retrieval, SQL Server has a special data structure called indexes.
- An index is mostly created on one or more columns which are commonly used in the SELECT clause or WHERE clause.
- **There are two types of indexes in SQL Server:**
  Clustered Indexes, and Non-Clustered Indexes
- **Clustered Indexes:**
  The clustered index defines the order in which the table data will be sorted and stored. As mentioned before, a table without indexes will be stored in an unordered structure. When you define a clustered index on a column, it will sort data based on that column values and store it. Thus, it helps in faster retrieval of the data.

SQL Server - Indexes

- There can be only one clustered index on a table because the data rows can be stored in only one order.
- When you create a Primary Key constraint on a table, a unique clustered index is automatically created on the table.
- **Syntax:**

```
CREATE CLUSTERED INDEX <index_name>
ON [schema.]<table_name>(column_name [asc|desc]);
```

- **Non-Clustered Indexes:**
  SQL Server provides two types of indexes, clustered and non-clustered indexes. Here you will learn non-clustered indexes.
- The non-clustered index does not sort the data rows physically. It creates a separate key-value structure from the table data where the key contains the column values (on which a non-clustered index is declared) and each value contains a pointer to the data row that contains the actual value. It is similar to a textbook having an index at the back of the book with page numbers pointing to the actual information.

SQL Server - Indexes

- There can be 999 non-clustered indexes on a single table is 999. When you create a Unique constraint, a unique non-clustered index is created on the table.

Syntax:

```
CREATE NONCLUSTERED INDEX <index_name>
ON <table_name>(column)
```

- When you create a Primary Key constraint on a table, a unique clustered index is automatically created on the table.

Syntax:

```
CREATE CLUSTERED INDEX <index_name>
ON [schema.]<table_name>(column_name [asc|desc]);
```

SQL Server - Joins

- SQL Server joins are essential for retrieving data from multiple tables in a relational database. They allow you to combine rows from two or more tables based on a related column between them.
- **1). INNER JOIN:** An INNER JOIN returns only the rows that have matching values in both tables.

```
SELECT column1, column2, ...
FROM table1
INNER JOIN table2
ON table1.column_name = table2.column_name;
```

- **2). LEFT JOIN (or LEFT OUTER JOIN):** A LEFT JOIN returns all rows from the left table and the matched rows from the right table. If there's no match, it returns NULL for columns from the right table.

```
SELECT column1, column2, ...
FROM table1
LEFT JOIN table2
ON table1.column_name = table2.column_name;
```

SQL Server - Joins

- **3). RIGHT JOIN (or RIGHT OUTER JOIN):** A RIGHT JOIN returns all rows from the right table and the matched rows from the left table. If there's no match, it returns NULL for columns from the left table.

```
SELECT column1, column2, ...
FROM table1
RIGHT JOIN table2
ON table1.column_name = table2.column_name;
```

- **4). FULL OUTER JOIN:** A FULL OUTER JOIN returns all rows when there is a match in either the left or the right table. If there's no match, it returns NULL for columns from the table without a match.

```
SELECT column1, column2, ...
FROM table1
FULL OUTER JOIN table2
ON table1.column_name = table2.column_name;
```

SQL Server - Joins

- **5). SELF JOIN:** A SELF JOIN is used to join a table with itself. It's often used when you have hierarchical or parent-child relationships within a table.

```
SELECT column1, column2, ...
FROM table1 AS t1
INNER JOIN table1 AS t2
ON t1.column_name = t2.column_name;
```

- **6). CROSS JOIN:** A CROSS JOIN returns the Cartesian product of two tables, meaning it combines every row from the first table with every row from the second table.

```
SELECT column1, column2, ...
FROM table1
CROSS JOIN table2;
```

Create Database

- In SQL Server, a database is made up of a collection of objects like tables, functions, stored procedures, views etc. Each instance of SQL Server can have one or more databases. SQL Server databases are stored in the file system as files.
- A login is used to gain access to a SQL Server instance and a database user is used to access a database. SQL Server Management Studio is widely used to work with a SQL Server database.
- **Type of Database in SQL Server**
  There are two types of databases in SQL Server: **System Database** and **User Database**.
- **System databases** are created automatically when SQL Server is installed. They are used by SSMS and other SQL Server APIs and tools, so it is not recommended to modify the system databases manually.
  The followings are the system databases:
  - master: master database stores all system level information for an instance of SQL Server. It includes instance-wide metadata such as logon accounts, endpoints, linked servers, and system configuration settings.

Create Database

- **The followings are the system databases:**
  - model: model database is used as a template for all databases created on the instance of SQL Server
  - msdb: msdb database is used by SQL Server Agent for scheduling alerts and jobs and by other features such as SQL Server Management Studio, Service Broker and Database Mail.
  - tempdb: tempdb database is used to hold temporary objects, intermediate result sets, and internal objects that the database engine creates.
- **User-defined Databases** are created by the database user using T-SQL or SSMS for your application data. A maximum of 32767 databases can be created in an SQL Server instance.
- You can execute the SQL script in the query editor using Master database.
- **Syntax:**

```
USE master;
CREATE <database-name>
```

Create Table

- Tables are database objects that contain all the data in a database. In a table, data is logically organized in rows and columns. Each column represents a field and each row represents a unique record. Each column has a data type associated with it. It represents the type of data in that column. Each table is uniquely named in a database.
- Number of tables in a database is only limited by the number of objects allowed in the database (2,147,483,647). A user-defined table can have up to 1024 columns.
- You can execute the CREATE TABLE statement in the query editor of SSMS to create a new table in SQL Server.
- **Syntax:**

```
CREATE TABLE [database_name.][schema_name.]table_name (
        pk_column_name data_type PRIMARY KEY,
        column_name2 data_type [NULL | NOT NULL],
        column_name3 data_type [NULL | NOT NULL],
        ...,
        [table_constraints]
);
```

Create Table

- Tables are database objects that contain all the data in a database. In a table, data is logically organized in rows and columns. Each column represents a field and each row represents a unique record. Each column has a data type associated with it. It represents the type of data in that column. Each table is uniquely named in a database.

Thank you